

Megatron in verl: Performance Tuning &FP8 rollout

Yan Bai, DevTech | verl meetup/20260110



Agenda

- The Evolution of Megatron–Core Integration
- Megatron Performance Tuning Methodology
- FP8 rollout
- NVIDIA’s Onging work with verl

The Evolution of Megatron–Core Integration

The Integration Evolution Journey

Early Stage: Config Converter

Core Idea

Write hyperparameter conversion logic for each individual model.

Drawbacks

Repetitive labor, inefficient, and not generalizable.

Process

Download model from HuggingFace → Offline conversion to Megatron's dist_checkpoint format → Load during training.

Middle Stage: mbridge

Core Idea

Abstract a unified bridge layer to decouple Verl and Megatron.

Process

Support direct online loading/storage of HuggingFace parameters, eliminating the need for offline conversion.

Advantages

Centralized the work of supporting new models into mbridge, greatly improving development efficiency.

The Integration Evolution Journey

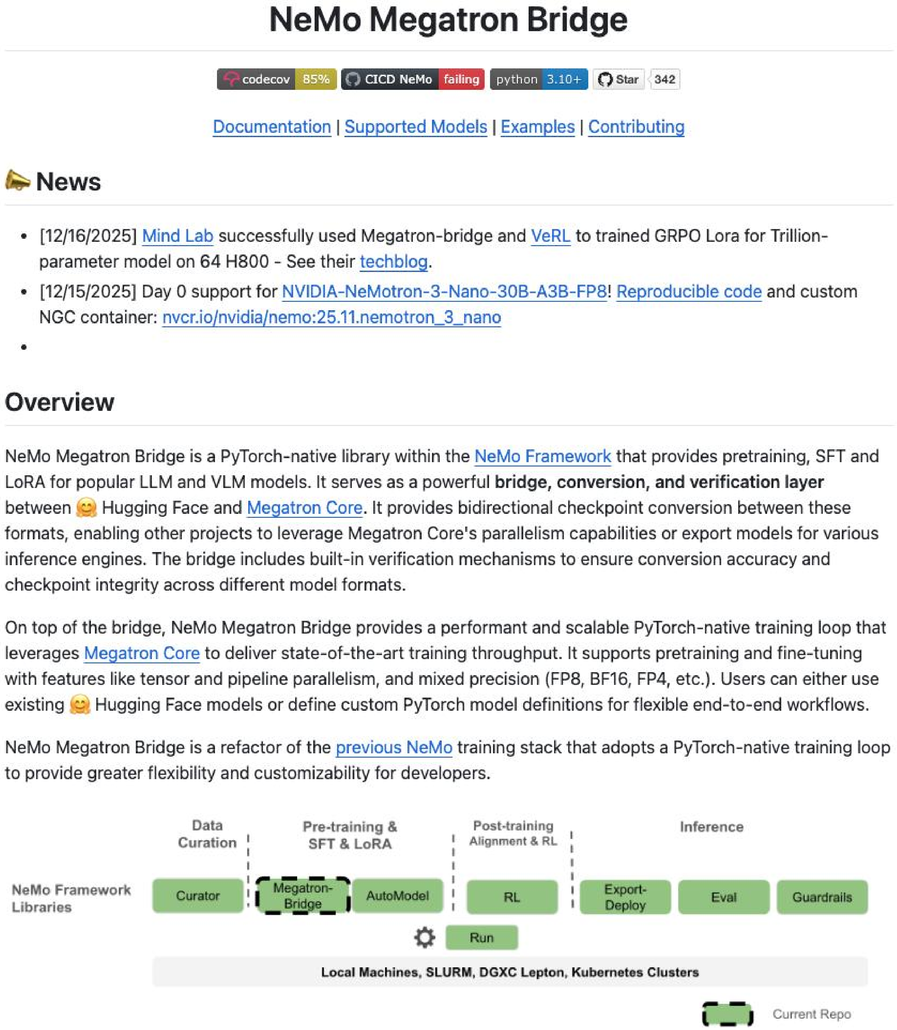
Present: Megatron-Bridge

Enhanced Features

Continuously supports new model architectures, multimodal capabilities, LoRA/PEFT, and other advanced training techniques.

Ecosystem Value

Serves as the official standard connecting the Verl ecosystem with NVIDIA's powerful training capabilities.



Megatron Performance Tuning Methodology

What is Megatron & When to Use It?

What is Megatron?

A large language model (LLM) training framework from NVIDIA.

Key Advantages: Scalability, efficiency, and performance for very large models.

When to Use Megatron?

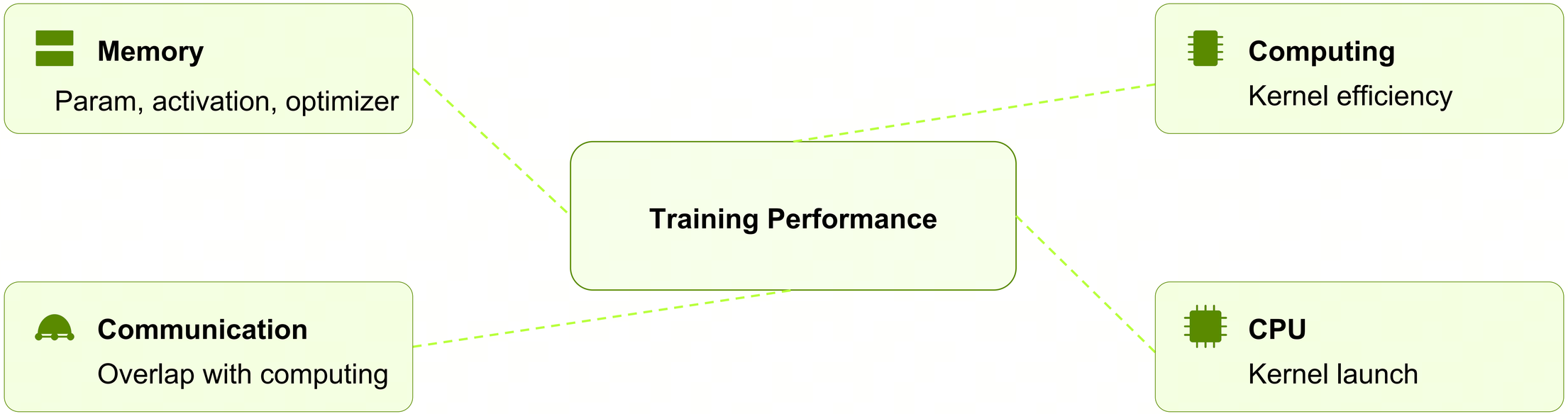
Consider Trying

Model parameters ≥ 14B, or number of GPUs ≥ 16, or sequence length ≥ 8k.

Highly Recommended

MoE models, or model parameters > 30B.

The Optimization Principle: A Balancing Act



Training performance tuning is essentially a multi-constraint optimization problem in a distributed environment.

Key Configurable Options in Megatron

Parallelism Strategy

- Tensor Parallelism (TP)
- Context Parallelism (CP)
- Pipeline Parallelism (PP)
- Virtual Parallelism(VPP)
- Expert Parallelism (EP)

n_GPU / PP = TP×CP×DP = EP×ExpertTP×ExpertDP

Re-computation Strategy

- Fully re-compute
- Selective with options [core_attn, moe_act, layernorm, mla_up_proj, mlp, moe, shared_experts]

Offloading Strategy

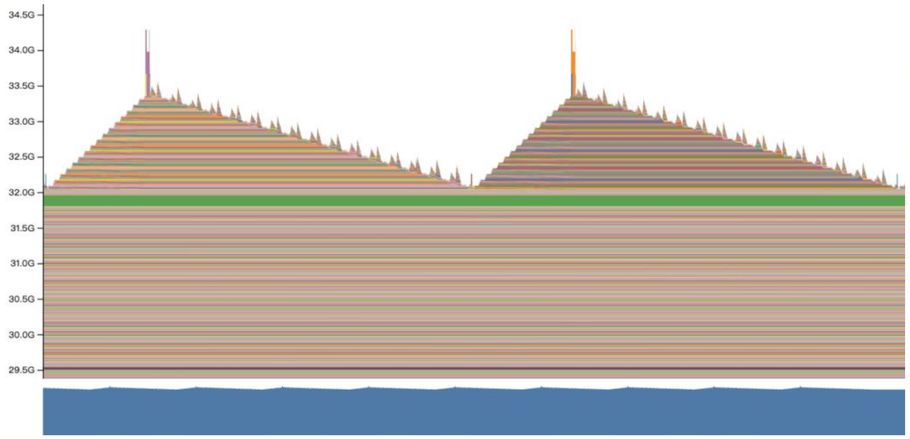
- Activation Offloading

* ExpertTP is usually set to 1

NVIDIA.

Step1: Play with the Memory Estimator

- Simulate the configuration with no costs
- Understanding how Parallelism and recompute affect the memory
- For more detail, see the official blog



Megatron Memory Estimator v0.13

Note: This estimator only measures the GPU memory directly managed by PyTorch when running Megatron. It does not include extra consumption from NCCL communication buffers, kernel fusion, overlap optimizations, CUDA Graphs, etc. Please use the "Overhead per GPU" option below to account for these additional costs.

Configuration

For detailed explanations of each parameter, please see the Megatron-LM arguments documentation.

Select a Local Config: Qwen/Qwen3-30B-A3B

Total GPUs: 32 | micro batch size: 1 | SeqLen: 10240

Use Distributed Optimizer

Recomputation: None

TP: 2 | PP: 1 | EP: 8 | CP: 1

VPP: None | ETP: 1

First Stage Layers: None | Last Stage Layers: None

Overhead per GPU: 10GB

Pipeline Layout (comma-separated layers per stage): e.g., Et|(tt|)*30L

Estimate

Model Config (Editable)


```
{
  "architectures": [
    "Qwen3MoeForCausalLM"
  ],
  "attention_bias": false,
  "attention_dropout": 0,
  "bos_token_id": 151643,
  "decoder_sparse_step": 1,
  "eos_token_id": 151645,
  "head_dim": 128,
  "hidden_act": "silu",
  "hidden_size": 2048,
  "initializer_range": 0.02,
  "intermediate_size": 6144,
  "max_position_embeddings": 40960,
  "max_window_layers": 48,
  "mlp_only_layers": [],
  "model_type": "qwen3_moe",
  "moe_intermediate_size": 768,
  "norm_topk_prob": true,
  "num_attention_heads": 32,
  "num_experts": 128,
  "num_experts_per_tok": 8,
  "num_hidden_layers": 48,
  "num_key_value_heads": 4,
  "output_router_logits": false,
  "rms_norm_eps": 0.000001,
  "rope_scaling": null,
  "rope_theta": 1000000,
  "router_aux_loss_coef": 0.001,
  "sliding_window": null,
  "tie_word_embeddings": false,
  "torch_dtype": "bfloat16",
  "transformers_version": "4.51.0",
  "use_cache": true,
  "use_sliding_window": false,
  "vocab_size": 151936
}
```


History

Model	Weight Gradient Optimizer (GB)	Activation (GB)	Total (GB/GPU)	Actions
Qwen3-30B-A3B GPUs: 32 TP2 EP8 Sequence: 1*10k Recompute: Selective (moe_act+layernorm)	35.18	23.18	68.35	Restore Detail Delete

Step2: Adjust the configuration with empirical guidance

General Rule

Smaller parallelism = better performance, but higher memory pressure. (TP1 > TP2 > TP4)

Mixed Parallelism

Often better than increasing a single dimension (e.g., TP4PP4 > TP16 or PP16).

Long Sequences

More benefit from TP and CP (TP first, then CP).

Large Models (EP & PP)

- Small EP(8), Large PP with fully recompute: Good for beginners, but more compute.
- Large EP(32), Small PP: Better peak performance, but requires more tuning.
  - Increasing TP*CP and replace fully recompute to fine-grained recompute, step by step.
  - Once `moe` is remove from recompute mudules, try enable 1f1b overlap to achieve best performance.

Rule of Thumb

seq_len/(TP*CP) > 1k to avoid CPU bottlenecks.

num_micro_batches_per_mini_bach > PP*2 to avoid to much bubbles.

Step2: Adjust the configuration with empirical guidance

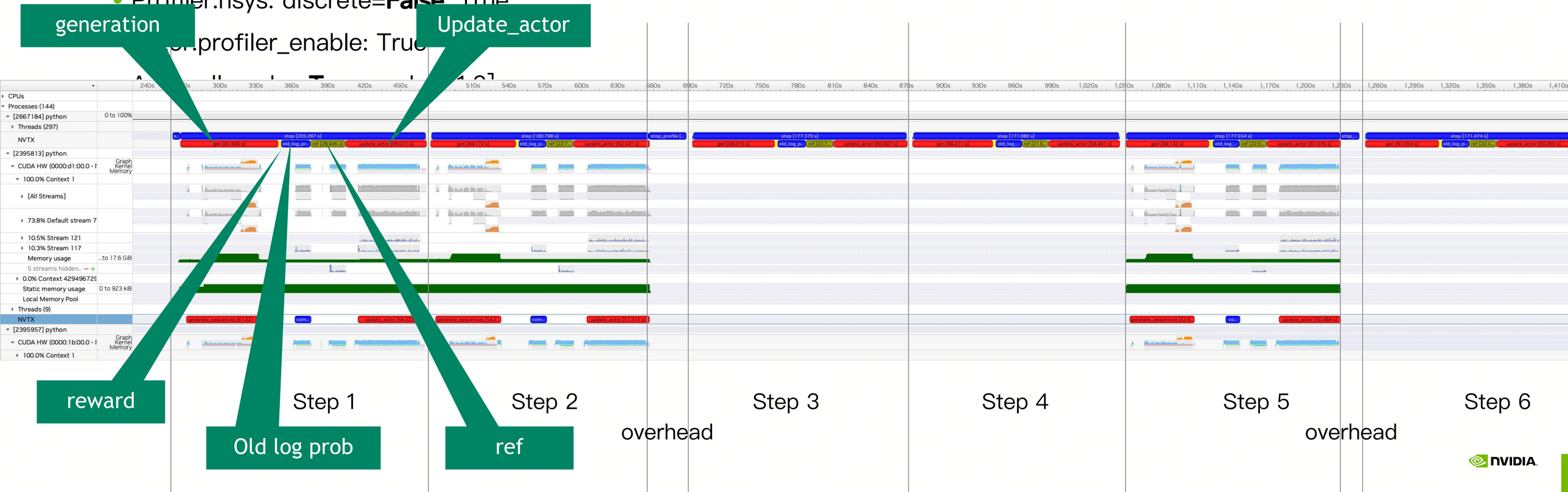
Takeaways

- Configuration tuning steps When encounter OOM, try increasing parallelism(TP*CP):
  - Fully recompute
  - Selective recompute with [layernorm, moe] (add mla_up_proj if MLA)
  - Selective recompute with [layernorm, moe_act] , Selective activation offloading with [expert_fc1, moe_act]
  - Selective recompute with [layernorm, moe_act]
- Enable DeepEP to to increase speed and reduce peak memory
- Enable RoPE fusion or lose up to 30% perf.
- Use CPU optimizer to offload the optimizer memory when forward and backward.
- Enable Dynamic Batching to make batches more even.
- Balance the data to reduce training bubbles caused by imbalanced workloads.
- Follow tuning practices

Step3: Profile & Iterate with Nsight System

- Profiler.tool: **nsys**
- Profiler steps: **[1,2, 5],** null, []
- Profiler.continuous_steps=**True**, False

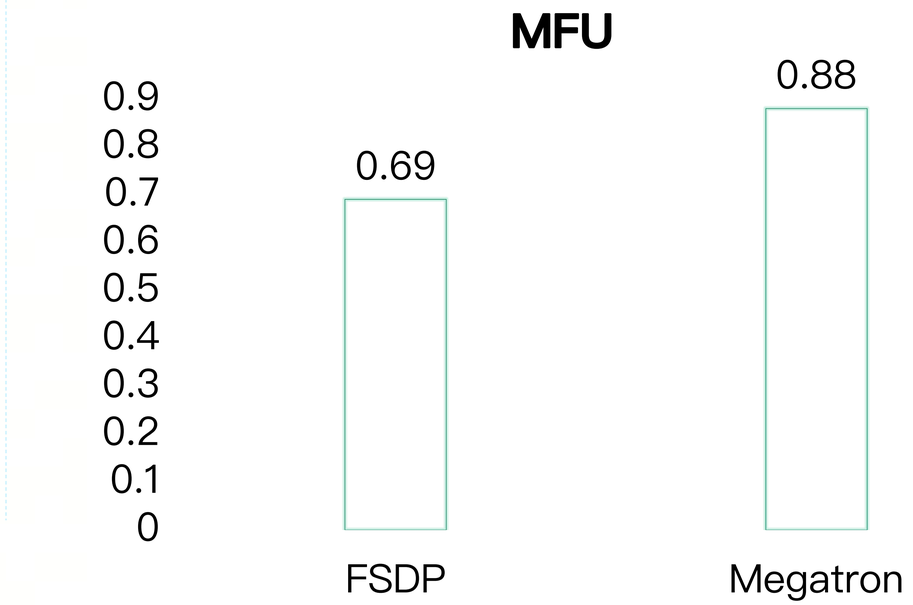
- Profiler.nsys. discrete=**False**, True
- [illegible].profiler_enable: True



Case Study 1: 14B Dense Model

Setup

32*H20, DAPO with 22k max seqlen

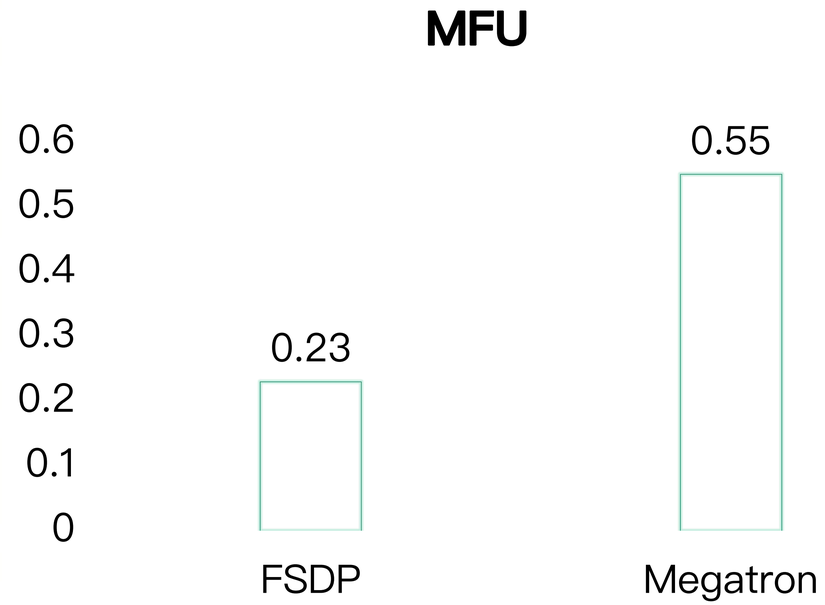


Megatron shows a **27.5%** improvement in training MFU over FSDP.

Case Study 2: 30B MoE Model

Setup

32*H20, DAPO with 22k max seqlen



Megatron shows a **2.39x** training MFU than FSDP.

Case Study 3: DeepSeek v3

Setup

512*H20, 128k max seqlen, Full Recomputation

parallelism	step time
PP16TP8CP2EP32	~50 mins
PP16TP4CP1EP32	~35 mins
PP16TP8CP1EP16	~30mins

Careful tuning of parallelism can lead to significant step time reduction.

NVIDIA.

Other Megatron Features for verl

FP8 Training

https://github.com/volcengine/verl/pull/4223

FP16 Training

https://github.com/volcengine/verl/pull/4086

Router Replay

https://github.com/volcengine/verl/pull/4101

LoRA

https://github.com/volcengine/verl/pull/4063

FP8 rollout

Xue Huang, Solution Architect

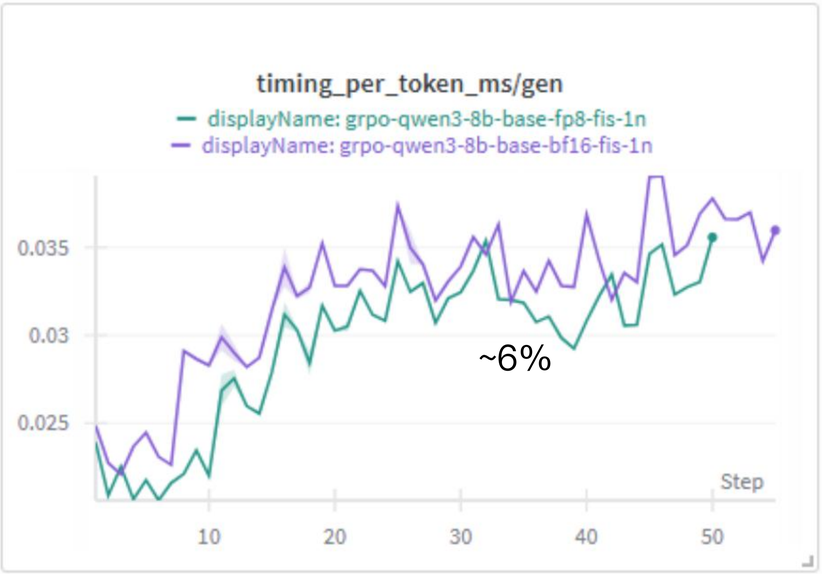
FP8 RL

- Options:
  - Rollout FP8, Training BF16 :
    - The time proportion of Rollout is high, we can achieve performance acceleration through FP8 with minimal impact on accuracy
    - Starting from this item
  - Rollout FP8, Training FP8
    - More complex, requiring attention to the impact on accuracy
  - Support both vLLM and SGLang

Qwen3-8B vllm benchmark

H200	avg_token_per_sec		speedup
	bf16	fp8	
固定输入长度，固定输出长度	9618.48	10242.76	1.06
固定输入长度，不固定输出长度	9658.61	10241.92	1.06
不固定输入长度，不固定输出长度	9894.17	10379.98	1.05

	max_output_len	max_seq	bf16	fp8	speedup
Qwen3-8B	2048	512	10463.90	10986.05	1.05
Qwen3-8B	2048	1024	9953.69	10492.53	1.05
Qwen3-8B	2048	2048	9796.85	10419.08	1.06
Qwen3-8B	4096	512	7180.02	7576.49	1.06
Qwen3-8B	4096	1024	7075.19	7329.25	1.04
Qwen3-8B	4096	2048	6926.40	7354.96	1.06



Qwen3 8B

TIS + partially quantization



- 在FP8的DAPO训练中，观察到与BF16的mismatch。FP8 rollout 往往会从训练一开始就产生更长的response。
- Ignore 前几层或后几层不进行量化：训练更stable
- DAPO Qwen3-8B-Base, prompt batch size 128*3. rollout n: 8

NVIDIA’s ongoing work with verl

On going works and roadmap

- TensorRT-LLM integration as an alternative rollout engine for performance, ready soon!
- Cool optimization features from megatron dev branch
  - Dynamic CP hot switch: adaptive alter the CP size according to sequence length
  - Muon: a magic optimizer to reduce up to 30% training steps and 50% optimizer memory
  - ……
- Megatron Local Parallel(M4): parallel states localized, supporting different parallel sets for ref/critic models
- Integrate more efficient multi-modal solution based on M4
- integrate Megatron-FSDP as an alternative backend, promising performance improvement on futher hardwards such as Grace-Blackwell
- integrate multi-modal solution based on Megatron-FSDP

NVIDIA.



欢迎扫码关注

NVIDIA 企业开发者社区

完成调研问卷，有机会获得NVIDIA精美礼品

礼品有限，先到先得

NVIDIA